Microjuego Asteroids

Fundamentos de Videojuegos - UPM

Autor: Jaime Calero

1. Escena y configuración 2D

El proyecto se organiza alrededor de MainScene.unity, una escena 2D con cámara ortográfica. La escala de los objetos no depende de la profundidad y el fondo espacial queda detrás de todas las entidades de juego.

Parámetro	Configuración
Cámara	Orthographic = true; Size = 5.0; posición (0, 0, -10).
Área visible	Altura de 10 unidades; en 16:9, ancho aproximado 17.77 (X = -8.88 a +8.88).
Clipping planes	Near = 0.3; Far = 1000.
Fondo	RGBA(13, 13, 20, 255). Background: SpriteRenderer con Space.jpg en (0, 0, 5), sortingOrder = -10.
Ejes	X horizontal; Y vertical; Z perpendicular a la pantalla. Las entidades interactivas usan Z = 0.

2. Assets y GameObjects

Objeto	Componentes / asset / función
Player	Transform, SpriteRenderer, Rigidbody, SphereCollider y Player.cs. Ship.png (529 x 577). Tag Player / Default.
Gun	Transform hijo de Player en X = +0.6. Origen de los disparos.
BulletPrefab	SpriteRenderer, SphereCollider Trigger y Bullet.cs. Bullet.png (4 x 4). Tag Bullet.
AsteroidPrefab	SpriteRenderer, SphereCollider Trigger y Asteroid.cs. Meteor.png (1000 x 1000). Tag Enemy.
MiniAsteroidPrefab	Meteor.png a escala 0.5x, SphereCollider y Asteroid.cs. Fragmento de una colisión.
EnemySpawner / ObjectPoolManager	Scripts EnemySpawner.cs y ObjectPool.cs. Generación y reciclaje de entidades.
Canvas / EventSystem	Canvas, CanvasScaler, GraphicRaycaster y UIManager.cs; EventSystem y StandaloneInputModule. HUD y entrada de UI.

La nave apunta naturalmente a Vector3.right. Todos los objetos de juego usan la capa Default salvo la interfaz, que usa UI.

3. Mecánicas de juego

3.1 Movimiento de la nave

La nave se mueve con Rigidbody.AddForce en la dirección transform.right. El código configura thrustForce = 6, rotationSpeed = 150 y linearDamping / drag = 1. El amortiguamiento mantiene la inercia al soltar el acelerador. El giro se hace directamente con transform.Rotate(Vector3.forward, -rotation), por lo que responde de forma inmediata.

- Envoltorio toroidal: si X > 9.0 o X < -9.0, o Y > 5.2 o Y < -5.2, la nave aparece en el extremo contrario.
- La física controla el desplazamiento; el script controla la orientación.
- La entrada de empuje negativa se anula: la nave solo acelera hacia delante, como en la mecánica tradicional de Asteroids.

3.2 Disparo

- Se dispara con barra espaciadora o Fire1, con un intervalo mínimo de 0.25 s entre disparos. El proyectil nace en Gun y hereda la orientación de la nave.
- Velocidad lineal: 12 m/s, usando targetVector.normalized * speed * Time.deltaTime.
- Vida máxima de la bala: 3 s. Con ObjectPool activo se desactiva y vuelve a la cola; sin piscina se destruye al terminar su vida.

3.3 Generación de enemigos

- Los meteoritos aparecen en Y = +6.2 y en una X aleatoria entre -8.0 y +8.0.
- La tasa inicial es de 30 asteroides por minuto y aumenta en 1 por minuto con cada generación.
- Intervalo de generación: 60 / spawnRatePerMinute.

3.4 Fragmentación angular (Iteración 2)

Cada colisión asteroide-bala produce 2 mini-asteroides si el objeto golpeado es un asteroide principal. El vector de la bala es la bisectriz de las dos trayectorias. Con separationAngle = 35 grados:

$$\theta_b = \operatorname{atan2}(v_y, v_x)$$
$$\theta_1 = \theta_b + 35 \quad \theta_2 = \theta_b - 35$$
$$u_1 = (\cos(\theta_1), \sin(\theta_1)) \quad u_2 = (\cos(\theta_2), \sin(\theta_2))$$

Los fragmentos se obtienen del pool de mini-asteroides y reciben la dirección calculada. Su velocidad es speed * 1.3; con el valor base actual de 2.5, equivale a 3.25 m/s. Tienen escala 0.5x en el prefab. Un impacto posterior los desactiva sin generar nuevos fragmentos. Cada impacto de bala suma 10 puntos.

4. Interfaz y controles

Acción	Primaria	Alternativa / interfaz
Rotar izquierda	Flecha izquierda	A
Rotar derecha	Flecha derecha	D
Empuje	Flecha arriba	W
Disparo	Espacio	Fire1 / clic izquierdo
Pausa / reanudar	Escape	Cancel / botón Pausa

5. Arquitectura software

La arquitectura separa el control del jugador, el spawn, la interfaz, el combate y la gestión de memoria. GameManager coordina el estado general del juego.

Script	Responsabilidad
Player.cs	Rigidbody, ejes Rotate y Thrust, entradas estándar, giro, disparos, límites y variable Player.SCORE.
Bullet.cs	Movimiento, temporizador, OnTriggerEnter(Collider), detección de Enemy, +10 puntos y refresco de UI.
Asteroid.cs	Movimiento descendente, rotación, TakeHit(Vector3 bulletDirection), fragmentación, colisión con Player y reinicio.
EnemySpawner.cs	Spawn temporal acelerado e invocación de objetos desde la piscina.
ObjectPool.cs	Singleton con colas y métodos GetBullet, GetAsteroid, GetMiniAsteroid y sus retornos.
GameManager.cs	Estados Jugando, Pausado y Game Over; Time.timeScale; tecla Escape y botones.
UIManager.cs	Texto de puntuación y visibilidad de los paneles de pausa y derrota.

Relación principal: GameManager coordina Player, EnemySpawner y UIManager. Player se relaciona con Bullet; Bullet y Asteroid resuelven los impactos; las entidades se apoyan en ObjectPool.

6. Pausa y reinicio

La pausa se activa con Escape o con el botón superior derecho. Al pausar, Time.timeScale = 0f y se congelan las físicas, las balas, los meteoritos y el contador del spawner.

Opción	Resultado
Reanudar	Time.timeScale = 1f y se oculta el panel.
Reiniciar	Marcador a 0, tiempo normal y recarga de escena mediante SceneManager.LoadScene().
Salir	Cierra la aplicación o termina el modo Play en el editor de Unity.

7. Flujo de una partida

- El jugador rota y acelera dentro de una pantalla toroidal.
- EnemySpawner crea meteoritos progresivamente desde la parte superior.
- El jugador dispara; Bullet detecta Enemy, actualiza el marcador y devuelve entidades a la piscina.
- Asteroid gestiona el impacto: el primario puede crear dos fragmentos con separación angular.
- Una colisión con Player notifica la muerte de la nave y reinicia la escena.

8. Object Pooling y rendimiento

ObjectPool.cs prepara en Awake una reserva de objetos inactivos y los reutiliza durante la partida. Esto reduce las llamadas repetidas a Instantiate y Destroy. Si una cola se agota, el propio código crea un objeto adicional bajo demanda, por lo que el pool es extensible y no garantiza una tasa de asignación estrictamente nula.

Pool	Reserva inicial	Funcionamiento
Balas	25	GetBullet: extraer, activar y posicionar. ReturnBullet: desactivar y reencolar.
Asteroides	15	GetAsteroid y ReturnAsteroid siguiendo el mismo ciclo.
Mini-asteroides	30	GetMiniAsteroid y ReturnMiniAsteroid.

Ciclo de reutilización

1) Queue.Dequeue extrae un objeto disponible. 2) Se activan sus componentes y se coloca en escena. 3) Tras su ciclo de vida o un impacto se hace SetActive(false). 4) Se reparenta al contenedor inactivo y se ejecuta Queue.Enqueue. Cuando la cola está vacía, GetFromQueue llama a CreateNewObject para ampliar la piscina.

9. Resumen de implementación

Área	Criterio
Escena	MainScene carga con cámara ortográfica, fondo en sortingOrder -10 y entidades en Z = 0.
Movimiento	Rigidbody, thrustForce 6, rotationSpeed 150, drag 1 y wrapping en X = 9.0 / Y = 5.2.
Disparo	Bala en Gun, 12 m/s, intervalo de 0.25 s, vida de 3 s y retorno al pool.
Spawn	Y = 6.2, X aleatoria entre -8 y +8, inicio en 30 por minuto e incremento de 1.
Combate	+10 por impacto; fragmentación de primarios con bisectriz a 35 grados; fragmentos a speed * 1.3.
Estados	Jugando, Pausado y Game Over; Escape, Cancel, botones y reinicio funcionales.
Interfaz	Score, botón de pausa, paneles modales y EventSystem.

Conclusión

El proyecto propone un microjuego sencillo de entender y suficientemente completo para demostrar fundamentos de Unity: escena 2D, entrada, física 3D con compatibilidad de colisiones 2D, UI, estados, generación procedural y reutilización de objetos. La segunda iteración añade profundidad al combate y control del flujo de la partida manteniendo la base del diseño.